TUTORIAL DE GIT · CAPÍTULO 3

Ramificaciones en Git

Crear ramas, fusionarlas, resolver conflictos y trabajar con ramas remotas

1. ¿Qué es una rama?

Una **rama** es un puntero móvil y ligero que apunta a un commit. La rama por defecto se llama `main` (históricamente `master`). Con cada nuevo commit, el puntero de la rama actual avanza automáticamente. **HEAD** es un puntero especial que indica en qué rama se está trabajando.

Como una rama es solo un archivo de 41 bytes (el hash del commit), crearlas y cambiarse entre ellas es casi instantáneo. Esto fomenta flujos de trabajo con muchas ramas pequeñas.

2. Crear y cambiar de rama

```
git branch prueba               # crea la rama (no cambia a ella)
git switch prueba               # cambia a la rama
git switch -c prueba            # crea y cambia en un solo paso
git checkout -b prueba          # equivalente clásico
git log --oneline --decorate --graph --all   # ver dónde apunta cada rama
```

3. Fusionar ramas (merge)

Para integrar el trabajo de una rama en otra, primero se cambia a la rama que **recibe** los cambios y luego se ejecuta `git merge`:

```
git switch main
git merge prueba
```

Tipo de fusión	Cuándo ocurre
Avance rápido (*fast-forward*)	La rama destino no tiene commits nuevos respecto a la rama que se fusiona; Git solo mueve el puntero hacia adelante.
Fusión a tres bandas (*three-way merge*)	Las historias divergieron; Git usa los dos extremos y su ancestro común para crear un commit de fusión con dos padres.

Conflictos de fusión

Si ambas ramas modificaron la misma parte de un archivo, Git se detiene y marca el conflicto. Para resolverlo:

1. Ejecutar `git status` para ver los archivos en conflicto (*unmerged*).
2. Abrir cada archivo y decidir qué versión conservar, eliminando las marcas.
3. Marcar el archivo como resuelto con `git add archivo`.
4. Terminar con `git commit` para crear el commit de fusión.

```
<<<<<<< HEAD
versión de la rama actual
=======
versión de la rama que se fusiona
>>>>>>> prueba
```

También se puede usar una herramienta visual con `git mergetool`.

4. Gestión de ramas

```
git branch                  # listar ramas (* marca la actual)
git branch -v               # último commit de cada rama
git branch --merged         # ramas ya fusionadas en la actual
git branch --no-merged      # ramas con trabajo sin fusionar
git branch -d prueba        # borrar una rama ya fusionada
git branch -D prueba        # forzar el borrado (pierde el trabajo no
fusionado)
```

5. Flujos de trabajo con ramas

- **Ramas de larga duración:** ramas siempre abiertas, por ejemplo `main` (código estable) y `develop` (integración); el código avanza de la más inestable a la más estable.
- **Ramas puntuales o de tema** (*topic branches*): ramas de vida corta para una sola función o corrección, que se fusionan y se borran.

6. Ramas remotas

Las **ramas de seguimiento remoto** (por ejemplo `origin/main`) son referencias locales que reflejan el estado de las ramas en el servidor; se actualizan con `git fetch`, no se pueden mover manualmente.

```
git fetch origin                          # actualizar las referencias remotas
git push origin prueba                    # publicar una rama local
git push -u origin prueba                 # publicar y dejarla en seguimiento
git switch --track origin/prueba          # crear rama local que sigue a una
remota
git branch -vv                            # ver qué rama local sigue a cuál
remota
git push origin --delete prueba           # borrar una rama en el servidor
```

7. Rebase

`git rebase` toma los cambios confirmados en una rama y los reaplica sobre otra, produciendo una historia lineal en lugar de un commit de fusión.

```
git switch prueba
git rebase main          # reaplica los commits de prueba sobre main
git switch main
git merge prueba        # ahora es un avance rápido
```

> *Regla de oro del rebase: no rebasar commits que ya se publicaron fuera de tu repositorio, porque reescribe la historia y causa problemas a quienes ya la descargaron.*
>
> *Resumen: las ramas son punteros baratos; se crean con `git switch -c`, se integran con `git merge` (o `git rebase`) y se comparten con `git push`.*

Fuente: Adaptado de Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2 (licencia CC BY-NC-SA 3.0).